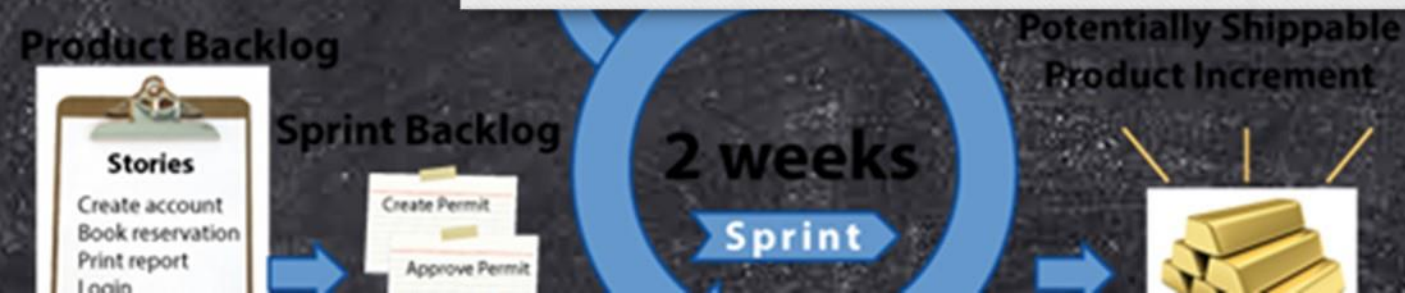


Desenvolvimento Ágil de Software

Professora: Cristiane Aparecida Lana



Desenvolvimento Ágil de Software

- Métodos ágeis
- Aplicabilidade dos Métodos Ágeis
- Técnicas de Desenvolvimento Ágil
- Gestão ágil de projetos
- Escalamiento de métodos ágeis
- Sugestões de ferramentas

Métodos Ágeis

Métodos Ágeis

- A insatisfação com o overhead* que envolve os métodos de projeto de software dos anos de 1980 e 1990 levou a criação de métodos ágeis. Esses métodos:
 - ✓ Têm foco no código ao invés de no projeto.
 - ✓ São baseados em uma abordagem iterativa de desenvolvimento de software.
 - ✓ São planejados para entregar rapidamente o software em funcionamento e evoluí-lo rapidamente para alcançar os requisitos em constante mudança.
- O objetivo dos métodos ágeis é reduzir o overhead nos processos de software (ex. limitando a documentação) e permitir uma resposta rápida aos requisitos em constante mudança sem retrabalho excessivo.

Manifesto Ágil (1990)

- Estamos **descobrimos melhores** formas de **desenvolver softwares** e ajudar **outros a fazê-lo também**. Através desse trabalho, valorizamos mais:
 - ✓ **Indivíduos e interações**, ao invés de **processos e ferramentas**.
 - ✓ **Softwares que já funcionam** ao invés de **documentação abrangente**.
 - ✓ **Colaboração do cliente** ao invés de **negociação contratual**.
 - ✓ **Resposta a mudanças** ao invés de **seguir um plano**.
- O que **significa** que **existe valor nos itens a direita**, mas que **valorizamos mais** os itens a esquerda.

Os Princípios dos Métodos Ágeis

Princípios	Descrição
Envolvimento do cliente	Os clientes devem estar intimamente envolvidos em todo o processo de desenvolvimento. Sua função é fornecer e priorizar novos requisitos de sistema e avaliar as iterações do sistema.
Entrega incremental	O software é desenvolvido em incrementos com o cliente especificando os requisitos a serem incluídos em cada incremento
Pessoas não processos	As habilidades da equipe de desenvolvimento devem ser reconhecidas e exploradas . Os membros da equipe devem desenvolver suas próprias maneiras de trabalhar sem processos prescritivos.
Abrace a mudança	Espera-se que os requisitos do sistema mudem e, assim, projete o sistema para acomodar essas mudanças .
Mantenha a simplicidade	Foco na simplicidade tanto no software que está sendo desenvolvido quanto no processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.

Aplicabilidade dos Métodos Ágeis

Aplicabilidade dos Métodos Ágeis

- Os métodos ágeis têm sido particularmente bem-sucedidos para dois tipos de desenvolvimento de sistemas:
 1. **Desenvolvimento de produto** onde uma empresa de software está desenvolvendo um pequeno ou produto de médio porte para venda.
 - Nesse caso, praticamente todos os produtos de software e aplicativos são agora desenvolvidos usando uma abordagem ágil.
 2. **Desenvolvimento de sistema personalizado** dentro de uma organização,
 - É necessário que o cliente tenha um compromisso claro para se envolver no processo de desenvolvimento
 - E em situações que tenham poucas partes interessadas externas e regulamentações que afetam o software.

Aplicabilidade dos Métodos Ágeis

Devido ao **foco em equipes pequenas e fortemente integradas**, pode existir **problemas na escalabilidade** de métodos ágeis em **sistemas grandes**.

Problemas com os Métodos Ágeis

- Pode ser difícil **manter o interesse dos clientes** que estão envolvidos no processo.
- Membros da **equipe podem não ser adequados ao envolvimento intenso** que caracteriza os métodos ágeis e **não interagir** bem com a equipe.
- **Priorizar mudanças** pode ser difícil onde existem **múltiplos stakeholders**.
- **Manter a simplicidade** requer trabalho extra.
- Os **contratos podem ser um problema** assim como em outras abordagens que usam o desenvolvimento iterativo.

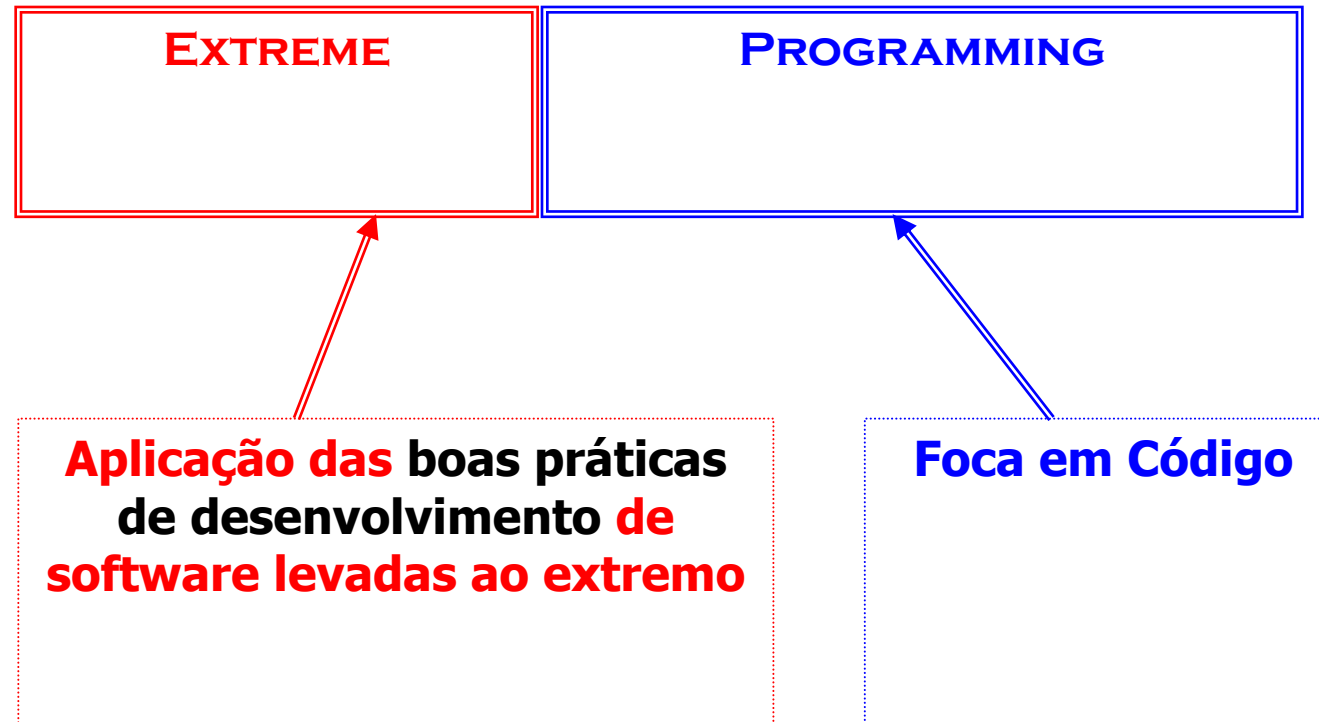
Métodos Ágeis e Manutenção de Software

- A maioria das organizações **gasta mais na manutenção** de softwares existentes do que **no desenvolvimento de softwares novos**.
- Devido a isso, para que os **métodos ágeis obtenham sucesso**, os softwares devem receber **atenção** tanto na **manutenção** quanto **no desenvolvimento original**.

Técnicas de Desenvolvimento Ágil

Programação Extrema – *Extreme Programming (XP)*

- Um método ágil muito influente, desenvolvido no final dos anos 1990, que introduziu uma variedade de técnicas de desenvolvimento ágil



Programação Extrema – *Extreme Programming*

- O XP talvez seja o método ágil mais conhecido e amplamente usado.
- O XP usa uma abordagem 'extrema' ao desenvolvimento iterativo.
 - ✓ Novas versões podem ser construídas várias vezes por dia;
 - ✓ Incrementos são entregues aos clientes a cada 2 semanas;
 - ✓ Todos os testes devem ser realizados em todas as versões;
 - ✓ Cada versão só é aceita se os testes forem concluídos com sucesso.

Os Princípios dos Métodos Ágeis e do XP

- O desenvolvimento incremental é mantido através de releases (novas versões do sistema) de sistema pequenos e frequentes.
- O envolvimento do cliente significa compromisso do cliente com a equipe em tempo integral.
- 'Pessoas e não processos' por meio de programação em pares, propriedade coletiva do código e um processo que evita longas horas de trabalho.
- Mudanças suportadas através de releases regulares de sistema.
- Manter a simplicidade através de constante refatoração de código.

Práticas do Extreme Programming (a)

Tabela 3.2 Práticas de Extreme Programming

Princípio ou prática	Descrição
Planejamento incremental	Os requisitos são gravados em cartões de estória e as estórias que serão incluídas em um release são determinadas pelo tempo disponível e sua relativa prioridade. Os desenvolvedores dividem essas estórias em 'Tarefas'. Veja os quadros 3.1 e 3.2.
Pequenos <i>releases</i>	Em primeiro lugar, desenvolve-se um conjunto mínimo de funcionalidades útil, que fornece o valor do negócio. <i>Releases</i> do sistema são frequentes e gradualmente adicionam funcionalidade ao primeiro <i>release</i> .
Projeto simples	Cada projeto é realizado para atender às necessidades atuais, e nada mais.
Desenvolvimento <i>test-first</i>	Um <i>framework</i> de testes iniciais automatizados é usado para escrever os testes para uma nova funcionalidade antes que a funcionalidade em si seja implementada.
Refatoração	Todos os desenvolvedores devem refatorar o código continuamente assim que encontrarem melhorias de código. Isso mantém o código simples e manutenível.

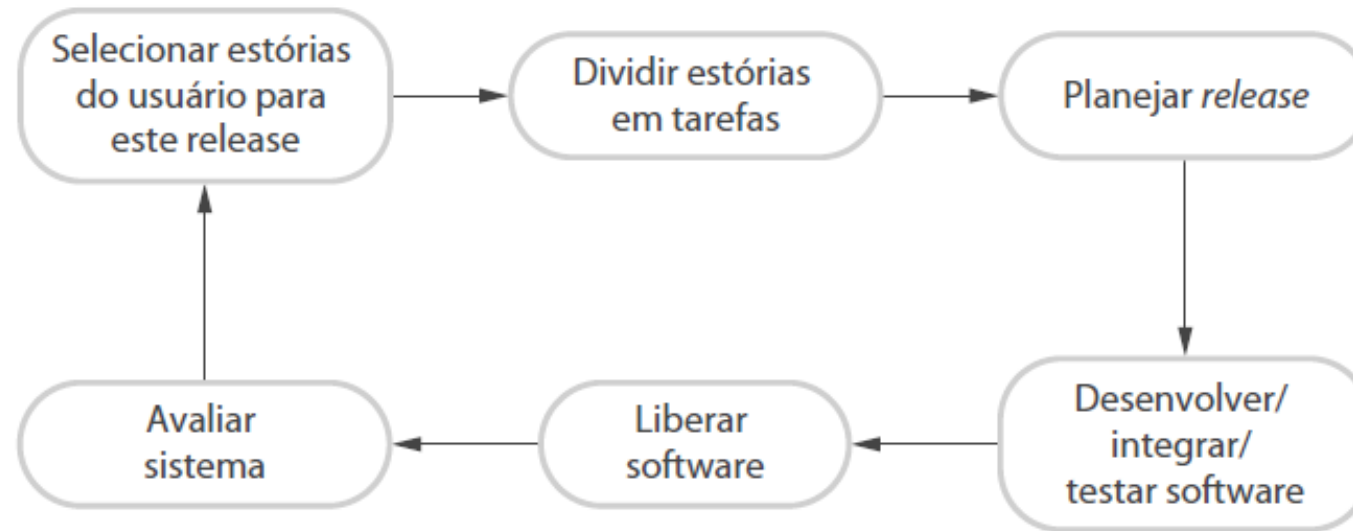
Práticas do Extreme Programming (b)

Programação em pares	Os desenvolvedores trabalham em pares, verificando o trabalho dos outros e prestando apoio para um bom trabalho sempre.
Propriedade coletiva	Os pares de desenvolvedores trabalham em todas as áreas do sistema, de modo que não se desenvolvam ilhas de <i>expertise</i> . Todos os conhecimentos e todos os desenvolvedores assumem responsabilidade por todo o código. Qualquer um pode mudar qualquer coisa.
Integração contínua	Assim que o trabalho em uma tarefa é concluído, ele é integrado ao sistema como um todo. Após essa integração, todos os testes de unidade do sistema devem passar.
Ritmo sustentável	Grandes quantidades de horas-extra não são consideradas aceitáveis, pois o resultado final, muitas vezes, é a redução da qualidade do código e da produtividade a médio prazo.
Cliente no local	Um representante do usuário final do sistema (o cliente) deve estar disponível todo o tempo à equipe de XP. Em um processo de Extreme Programming, o cliente é um membro da equipe de desenvolvimento e é responsável por levar a ela os requisitos de sistema para implementação.

O Ciclo de um Release em XP

Figura 3.2

O ciclo de um *release* em Extreme Programming



Práticas XP influentes

- Como já vimos em XP tem um foco técnico e não é fácil de integrar com a prática de gerenciamento na maioria das organizações.
- Assim, embora o desenvolvimento ágil use práticas do XP, o método originalmente definido não é amplamente utilizado.
- Práticas chave
 - Histórias de usuários para especificação
 - Reestruturação
 - Desenvolvimento test-first
 - Programação em pares

XP – Histórias de usuários para requisitos

- Em XP, um cliente ou usuário é parte do time de XP e é responsável na tomada de decisões sobre requisitos.
- Requisitos do usuário são expressos como cenários ou histórias dos usuários.
- Esses são escritos em cartões e a equipe de desenvolvimento os divide em tarefas de implementação. Essas tarefas são a base das estimativas de cronograma e custo.
- O cliente escolhe as histórias que serão incluídas no próximo release baseando-se nas suas prioridades e nas estimativas de cronograma.

Uma estória de 'Prescrição de Medicamentos'

Prescrição de medicamentos

Kate é uma médica que deseja prescrever medicamentos para um paciente de uma clínica. O prontuário do paciente já está sendo exibido em seu computador, assim, ela clica o campo 'medicação' e pode selecionar 'medicação atual', 'nova medicação', ou 'formulário'.

Se ela selecionar 'medicação atual', o sistema pede que ela verifique a dose. Se ela quiser mudar a dose, ela altera esta e em seguida, confirma a prescrição.

Se ela escolher 'nova medicação', o sistema assume que ela sabe qual medicação receitar.

Ela digita as primeiras letras do nome do medicamento. O sistema exibe uma lista de possíveis fármacos que começam com essas letras. Ela escolhe a medicação requerida e o sistema responde, pedindo-lhe para verificar se o medicamento selecionado está correto.

Ela insere a dose e, em seguida, confirma a prescrição.

Se ela escolhe 'formulário', o sistema exibe uma caixa de busca para o formulário aprovado.

Ela pode, então, procurar pelo medicamento requerido. Ela seleciona um medicamento e é solicitado que verifique se a medicação está correta. Ela insere a dose e, em seguida, confirma a prescrição.

O sistema sempre verifica se a dose está dentro da faixa permitida. Caso não esteja, Kate é convidada a alterar a dose.

Após Kate confirmar a prescrição, esta será exibida para verificação. Ela pode escolher 'OK' ou 'Alterar'. Se clicar em 'OK', a prescrição fica gravada nos bancos de dados da auditoria.

Se ela clicar em 'Alterar', reinicia o processo de 'Prescrição de Medicamentos'.

Exemplos de cartões de tarefa para a prescrição de medicamentos

- Resume **uma história**, reunindo as **necessidades do usuário**
- Os **cartões** devem **ser decompostos em tarefas** a **serem executadas** pelo time de desenvolvimento
- É nessa fase que o **esforço e os recursos** serão **identificados**
- Que as **histórias** serão **priorizadas**

Tarefa 1: Alterar dose de medicamentos prescritos

Tarefa 2: Seleção de formulário

Tarefa 3: Verificação de dose

A verificação da dose é uma precaução de segurança para verificar se o médico não receitou uma dose perigosamente pequena ou grande.

Usando o ID do formulário para o nome do medicamento genérico, procure o formulário e obtenha a dose mínima e máxima recomendada.

Verifique a dose mínima e máxima prescrita. Caso esteja fora da faixa, emita uma mensagem de erro dizendo que a dose está muito alta ou muito baixa.

Caso esteja dentro da faixa, habilite o botão 'Confirmar'.

XP e Mudança

- O senso comum da engenharia de software diz que se deve projetar pensando em mudanças
- Vale a pena gastar tempo e esforço antecipando as mudanças já que, posteriormente, esse esforço reduz custos no ciclo de vida.
- No entanto, o XP afirma que isso não vale a pena já que as mudanças não podem ser antecipadas de forma confiável.
- Ao invés disso, propõe melhorias constantes do código (refatoração) para tornar as mudanças mais fáceis quando for necessário a sua implementação.

Refatoração

- A equipe de programação busca possíveis melhorias de software e as faz mesmo quando essas não são uma necessidade imediata.
- O que melhora a inteligibilidade do software e reduz a necessidade de documentação.
- Torna-se mais fácil fazer mudanças porque o código é bem construído e limpo.
- No entanto, algumas mudanças requerem refatoração da arquitetura, o que é muito mais caro.

Exemplos de Refatoração

- Reorganização de uma hierarquia de classes para remover código duplicado.
- Organização e renomeação de atributos e métodos para torná-los mais fáceis de entender.
- A substituição do código com as chamadas para métodos definidos em uma biblioteca de programas.

Testes em XP

- Em XP, os testes são fundamentais
- XP desenvolveu uma abordagem em que o programa é testado depois de que cada alteração é feita.
- Características de testes em XP:
 1. Desenvolvimento *test-first*.
 2. Desenvolvimento de testes incrementais a partir de cenários.
 3. Envolvimento do usuário no desenvolvimento de testes e validação.
 4. Cada vez que um novo release é construído, são usados *frameworks* de testes automatizados para executarem todos os testes de componentes.

TDD – Test-Driven Development

- Desenvolvimento Dirigido a testes
 - Escrever testes antes do código esclarece os requisitos a serem implementados.
 - Os testes são escritos como programas em vez de dados para que possam ser executados automaticamente.
 - O teste inclui uma verificação de que foi executado corretamente.
 - Normalmente depende de uma estrutura de teste como o Junit.

TDD – Test-Driven Development

- Todos os testes anteriores e novos são executados automaticamente quando uma nova funcionalidade é adicionada, verificando assim se a nova funcionalidade não introduziu erros.

TDD – Test-Driven Development

- Envolvimento do Cliente
 - A função do cliente no processo de teste é ajudar a desenvolver testes de aceitação para as histórias que serão implementadas na próxima versão do sistema.
 - O cliente que faz parte da equipe escreve os testes à medida que o desenvolvimento avança.
 - Todo o novo código é, portanto, validado para garantir que é o que o cliente precisa.

Descrição de Caso de Teste para Verificação de Dose

Teste 4: Verificação de dose

Entrada:

1. Um número em mg representando uma única dose da medicação.
2. Um número que representa o número de doses únicas por dia.

Testes:

1. Teste para entradas em que a dose única é correta, mas a frequência é muito alta.
2. Teste para entradas em que a única dose é muito alta e muito baixa.
3. Teste para entradas em que a dose única x frequência é muito alta e muito baixa.
4. Teste para entradas em que a dose única x frequência é permitida.

Saída:

Mensagem de OK ou erro indicando que a dose está fora da faixa de segurança.

Dificuldades dos testes em XP

- Os **programadores preferem programar a testar** e as vezes eles usam atalhos quando escrevem esses testes.
 - Por exemplo, eles podem **escrever testes incompletos** que **não avaliam** todas **as possíveis exceções** que podem ocorrer.
- **Alguns testes** podem **ser muito difíceis** de **serem escritos** de forma incremental.
 - Por exemplo, em **uma interface** de usuário **complexa**, geralmente é **difícil escrever testes de unidade** para **o código** que implementa **a 'lógica de display'** e o fluxo de trabalho entre telas.
- É **difícil julgar** se um conjunto de testes está completo.
- Embora você tenha vários testes de sistema, **o conjunto dos testes pode não prover uma cobertura completa.**

Dificuldades dos testes em XP

- As pessoas que adotam a função de cliente têm tempo disponível limitado.
- Portanto, não podem trabalhar em tempo integral com a equipe de desenvolvimento.
- Eles podem sentir que fornecer os requisitos foi uma contribuição suficiente e, portanto, podem relutar em se envolver no processo de teste.

Ponto Importante

- A decisão de quando usar uma abordagem ao desenvolvimento ágil ou dirigida a planos deve depender do tipo de software que está sendo desenvolvido, da capacidades da equipe de desenvolvimento e da cultura da organização desenvolvedora do sistema.

Programação em pares

- A programação em pares envolve programadores desenvolvendo código juntos.
- Isso ajuda a desenvolver a propriedade comum do código e a disseminar o conhecimento por toda a equipe.
- Ele serve como um processo de revisão informal, pois cada linha de código é examinada por mais de 1 pessoa.

Programação em pares

- Ele **incentiva** a **refatoração**, pois toda a **equipe** pode **se beneficiar** com o aprimoramento do **código do sistema**.
- Na programação em par, os **programadores sentam-se juntos** no mesmo **computador** para **desenvolver o software**.
- Os **pares** são criados **dinamicamente** para **que todos os membros** da equipe **trabalhem uns com os outros** durante o **processo de desenvolvimento**.

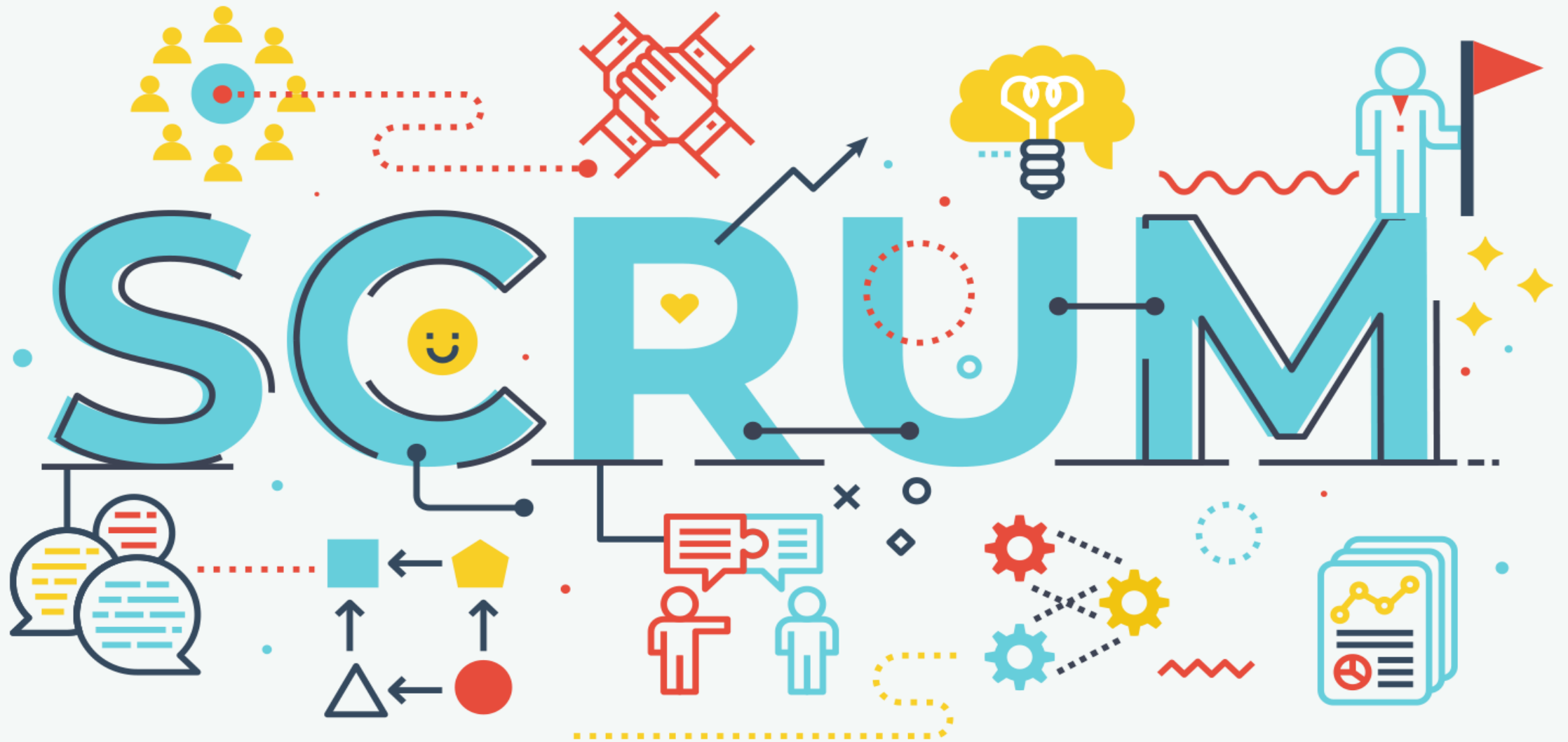
Programação em pares

- O compartilhamento de conhecimento que acontece durante a programação em par é muito importante, pois reduz os riscos gerais para um projeto quando os membros da equipe saem.
- A programação em pares não é necessariamente ineficiente
- Há algumas evidências que sugerem que um par trabalhando junto é mais eficiente do que 2 programadores trabalhando separadamente.

Gestão Ágil de Projetos

Cenários de requisitos

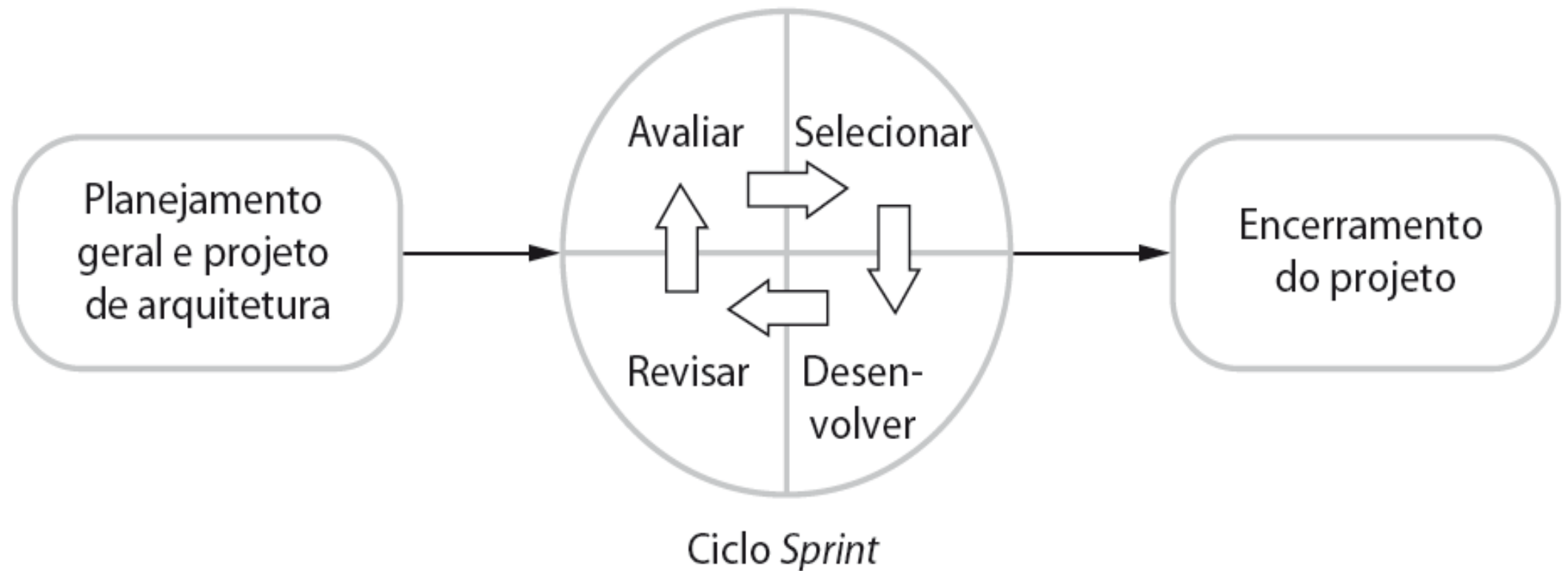
- A principal responsabilidade de gerentes de projeto de software é gerenciar o projeto para que o software seja entregue em tempo e dentro do orçamento planejado para o projeto.
- A abordagem padrão para o gerenciamento de projeto é dirigida a planos.
- Os gerentes estruturam um plano para o projeto mostrando o que deve ser entregue, quando deve ser entregue e quem irá trabalhar no desenvolvimento dos entregáveis (“*deliverables*”).
- O gerenciamento ágil de projetos requer uma abordagem diferente, adaptada ao desenvolvimento incremental e aos pontos fortes particulares dos métodos ágeis.



Scrum

- A abordagem Scrum é um método ágil genérico mas seu foco é na gerência de desenvolvimento iterativo ao invés de práticas ágeis específicas.
- Sobre o Scrum:
 1. Obtém-se do **Product Backlog** o que é de mais valor.
 2. Planeja-se a **iteração**.
 3. Faz-se **acompanhamento diário**.
 4. Entrega **acréscimo de funcionalidades** ao fim da iteração.

Scrum



Time Scrum

- Consiste em um Scrum Master, um Product Owner e Desenvolvedores do projeto.

- É uma unidade coesa de profissionais focados em **um objetivo de cada vez**, o **Objetivo do Produto/Processo/Serviço**.

- Os Times são multifuncionais, o que significa que os membros têm todas as habilidades necessárias para criar valor em cada Sprint.

- Eles também são autogerenciados, o que significa que decidem internamente quem faz o quê, quando e como.

Time Scrum

- O Time Scrum é responsável por todas as atividades relacionadas ao produto:
 - desde a colaboração das partes interessadas,
 - verificação,
 - manutenção,
 - operação,
 - experimentação,
 - pesquisa e desenvolvimento e
 - qualquer outra coisa que possa ser necessária.

Desenvolvedores

- Os **desenvolvedores** são sempre **responsáveis** por:
 - Criação de **um plano para o Sprint**, o **Sprint Backlog**;
 - Incutir **qualidade aderindo** a uma **Definição de Pronto**¹;
 - Adaptar o **plano a cada dia** em **direção ao objetivo** do Sprint; e,
 - Responsabilizando-se **mutuamente como profissionais**.

¹A Definição de Pronto é **uma descrição formal do estado do Incremento** quando **atende as medidas de qualidade** exigidas **para o produto**.

O Product Owner

- O Product Owner também é responsável pelo gerenciamento eficaz do Product Backlog, que inclui:
 - Desenvolver e comunicar explicitamente o **Objetivo do Produto**;
 - Criar e comunicar claramente os **itens do Product Backlog**;
 - Encomendar **itens do Product Backlog**; e,
 - Garantir que o **Product Backlog** seja **transparente, visível e compreendido**.

O Scrum Master

- O Scrum Master **serve o Time Scrum** de várias maneiras, incluindo:
 - **Treinar os membros** da equipe em **autogestão** e **multifuncionalidade**;
 - Ajudar o **Time Scrum** a se **concentrar** na **criação de Incrementos** de **alto valor** que atendam à **Definição de Pronto**;
 - Causar **a remoção de impedimentos** ao progresso do Time Scrum; e,
 - Garantir que **todos os eventos do Scrum** ocorram e **sejam positivos, produtivos** e mantidos **dentro do timebox**.

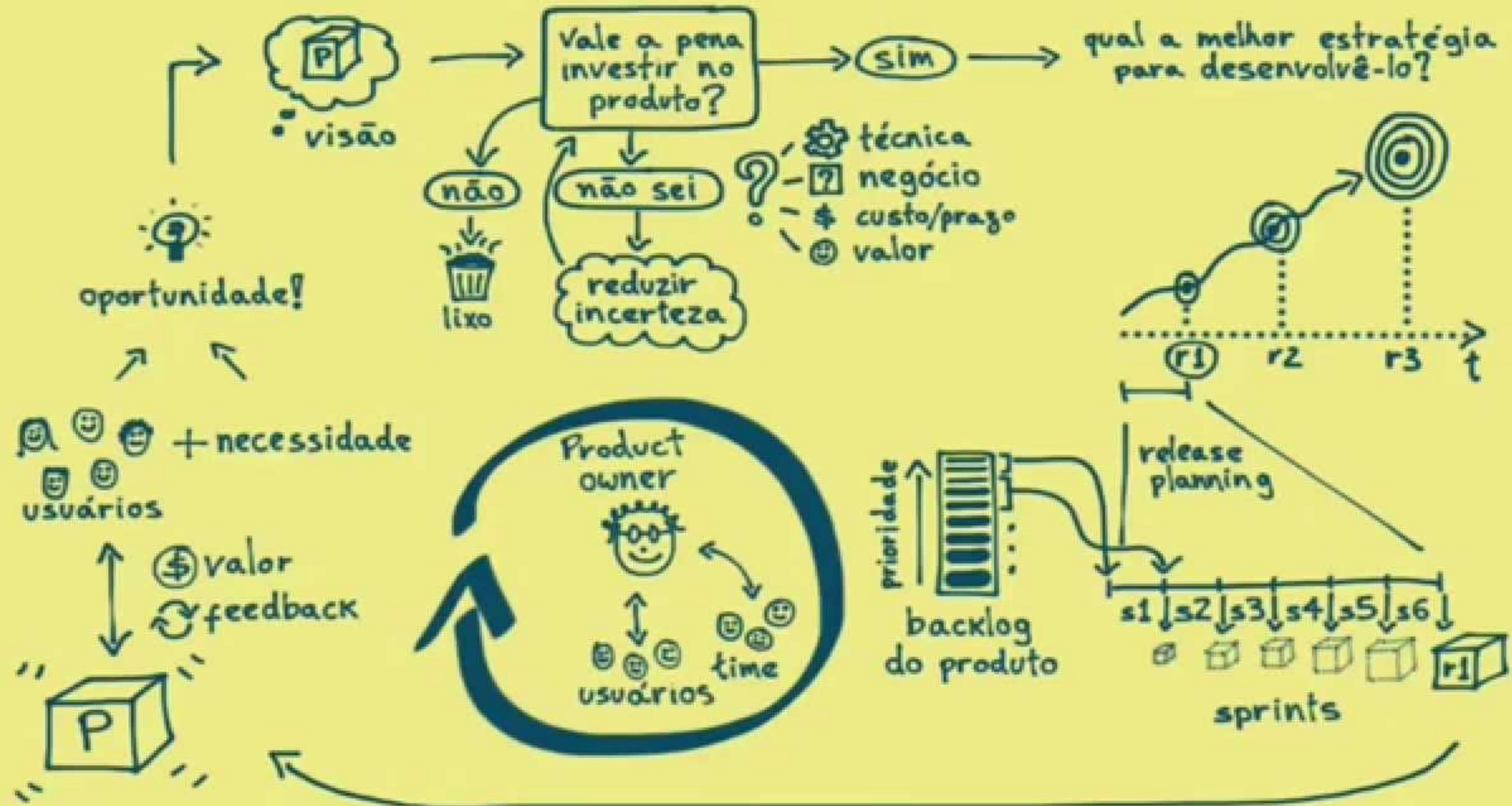
O Scrum Master

- O Scrum Master **serve a organização** de várias maneiras, incluindo:
 - Liderar, treinar a organização na adoção do Scrum;
 - Planejar e aconselhar implementações de Scrum dentro da organização;
 - Ajudar **funcionários e partes interessadas** a **entender** e **adotar** uma abordagem empírica para trabalhos complexos; e,
 - Remoção de barreiras entre as **partes interessadas** e as **equipes Scrum**.

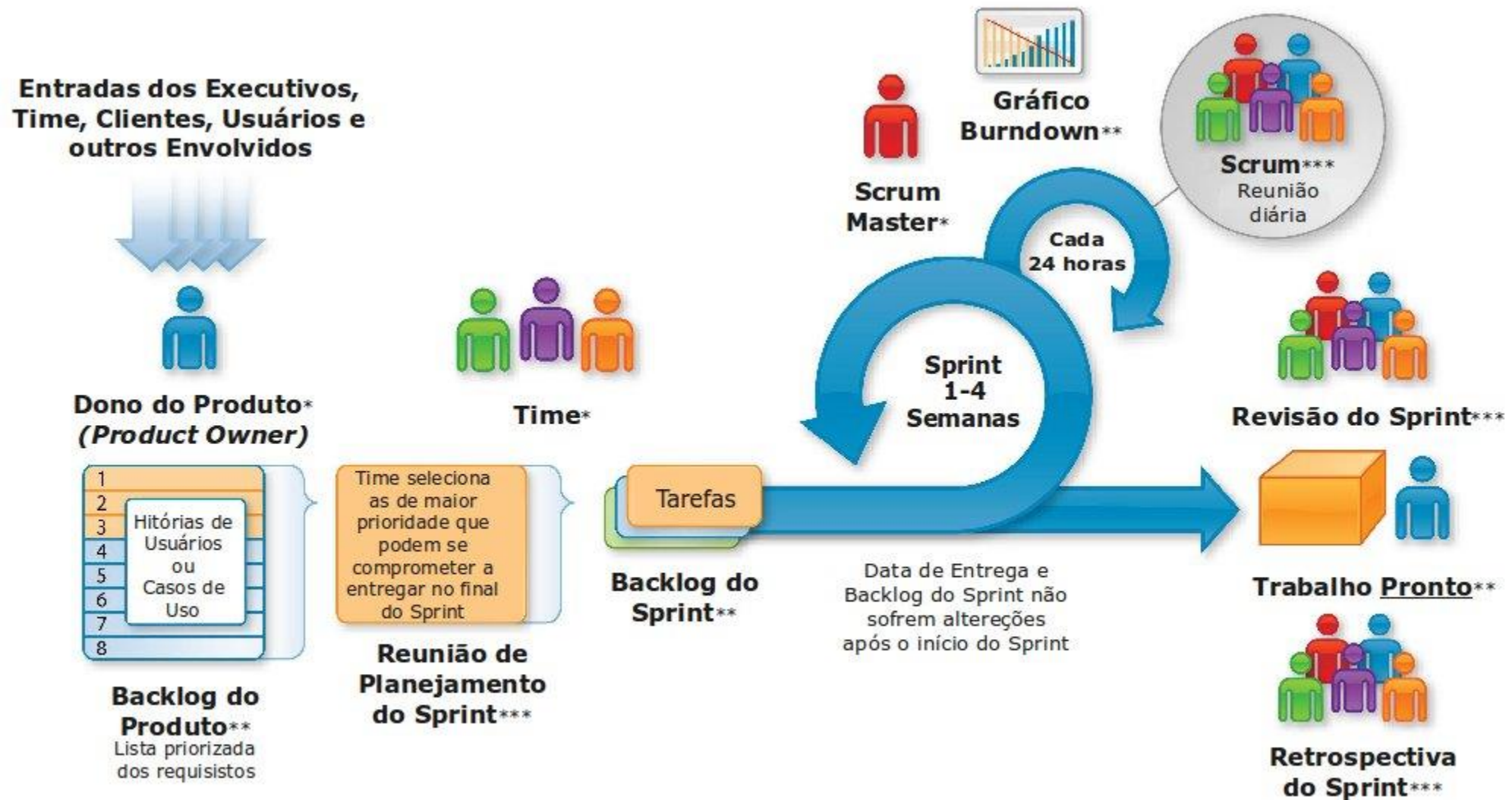
Artefatos do Scrum

- Cada etapa tem um artefato que contribui para a criação final de valor do produto desenvolvido na sprint
 - Para o Product Backlog é o Product Goal.
 - Para o Sprint Backlog é o Sprint Goal.
 - Para o Incremento é a Definição de Pronto.
- Esses compromissos existem para reforçar o empirismo e os valores do Scrum para o Time Scrum e seus stakeholders.

Scrum



Scrum



Scrum

Termo Scrum	Definição
Equipe de desenvolvimento (<i>Development Team</i>)	Um grupo auto-organizado de desenvolvedores de software, que não deve ter mais de 7 pessoas. Eles são responsáveis por desenvolver o software e outros documentos essenciais do projeto.
Incremento de produto potencialmente entregável (<i>Potentially shippable product increment</i>)	O incremento de software que é entregue a partir de uma sprint. A ideia é que isso deve ser 'potencialmente transportável', o que significa que está em um estado acabado e nenhum outro trabalho, como teste, é necessário para incorporá-lo ao produto final. Na prática, isso nem sempre é possível.
Backlog do produto (<i>Product backlog</i>)	Esta é uma lista de itens a fazer que a equipe Scrum deve enfrentar. Eles podem ser definições de recursos para o software, requisitos de software, histórias de usuário ou descrições de tarefas suplementares que são necessárias, como definição de arquitetura ou documentação do usuário.
Proprietário do produto (<i>Product owner</i>)	Um indivíduo (ou possivelmente um pequeno grupo) cujo trabalho é identificar os recursos ou requisitos do produto, priorizá-los para desenvolvimento e revisar continuamente a lista de pendências do produto para garantir que o projeto continue atendendo às necessidades críticas de negócios. O Product Owner pode ser um cliente, mas também pode ser um gerente de produto em uma empresa de software ou outro representante das partes interessadas.

Scrum

Termo Scrum	Definição
Reunião diária (<i>daily meeting or Scrum meeting</i>)	Uma reunião diária da equipe Scrum que analisa o progresso e prioriza o trabalho a ser feito naquele dia. Idealmente, esta deve ser uma pequena reunião presencial que inclua toda a equipe.
Scrum Master	O ScrumMaster é responsável por garantir que o processo Scrum seja seguido e orientar a equipe no uso eficaz do Scrum. Ele ou ela é responsável por fazer a interface com o resto da empresa e por garantir que a equipe Scrum não seja desviada por interferência externa. Os desenvolvedores do Scrum são inflexíveis quanto ao fato de o ScrumMaster não ser considerado um gerente de projeto. Outros, no entanto, podem nem sempre achar fácil ver a diferença.
Sprint	Uma iteração de desenvolvimento. Sprints geralmente duram de 2 a 4 semanas.
Velocidade (<i>Velocity</i>)	Uma estimativa de quanto esforço de backlog do produto que uma equipe pode cobrir em um único sprint. Compreender a velocidade de uma equipe os ajuda a estimar o que pode ser coberto em um sprint e fornece uma base para medir o desempenho aprimorado.

O ciclo de Sprint

- Os **Sprints** possuem um deadline definido, **geralmente de 2 a 4 semanas**.
- O ponto de partida de planejamento é o **backlog de produto**, que é a **lista de trabalho** a ser feito no projeto.
- A **fase de seleção** envolve a **seleção das características e funções** que **serão desenvolvidas** durante o Sprint, pela **equipe do projeto** que trabalha com o cliente.

O ciclo de Sprint

- Assim **que isso é definido**, a **equipe se organiza** para **desenvolver o projeto**.
- Durante esse estágio, a **equipe é isolada do cliente e da organização**, com **todas as comunicações canalizadas por meio do chamado “Scrum Master”**.
- A **função do Scrum Master** é proteger a **equipe de desenvolvimento** de **distrações externas**.
- Ao final do Sprint, **o trabalho feito é revisto** e apresentado **aos stakeholders**.
- Assim o **próximo ciclo de Sprint** começa.

Trabalho em equipe no Scrum

- O Scrum Master é um facilitador que organiza reuniões diárias, mantém o backlog do trabalho a ser feito, grava decisões, mede o processo usando o backlog e comunica-se com os clientes e a gerência fora da equipe.
- A equipe inteira comparece às reuniões diárias curtas nas quais todos os membros da equipe compartilham informações, descrevem seu progresso desde a última reunião, descrevem os problemas que surgiram e o que está planejado para o dia seguinte.
- ✓ Com isso, todos na equipe sabem o que está acontecendo e, caso ocorra um problema, podem replanejar o trabalho a curto prazo para lidar com a situação.

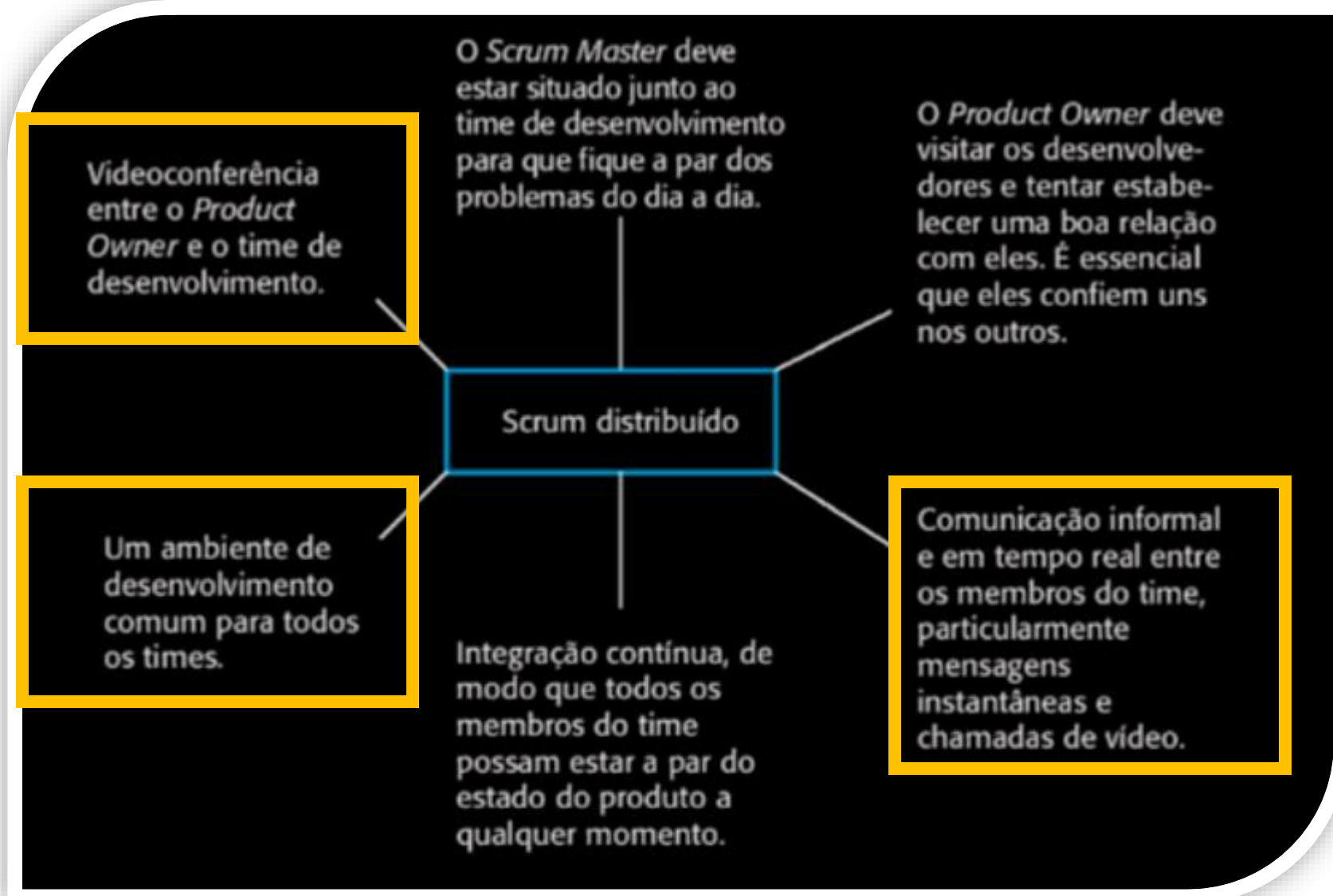
Benefícios do Scrum

- O produto é dividido em um conjunto de partes gerenciáveis e inteligíveis.
- Requisitos instáveis não impedem o progresso.
- Toda a equipe tem visão de tudo e conseqüentemente a comunicação da equipe é melhorada.

Benefícios do Scrum

- Os clientes recebem a entrega dos incrementos no tempo certo, além do feedback de como o produto funciona.
- Se estabelece a confiança entre os clientes e os desenvolvedores e se cria uma cultura positiva na qual todos acham que o projeto dará certo.
- O copo sempre está meio cheio e nunca meio vazio.

Scrum Distribuído



Escalamento dos Métodos Ágeis

Escalamento de Métodos Ágeis

- Aumentar a escala dos métodos ágeis envolve mudá-los para que lidem com projetos maiores e mais longos onde existem múltiplas equipes de desenvolvimento, talvez trabalhando em localizações diferentes.
-
- Isso porque o desenvolvimento de sistemas de software de grande porte é diferente de sistemas pequenos, em vários pontos, como vemos a seguir.

1 - Escalamento de Métodos Ágeis – Grandes Sistemas

- Sistemas de grande porte geralmente são coleções de sistemas separados que se comunicam, nos quais equipes separadas desenvolvem cada um dos sistemas.
- Frequentemente, essas equipes estão trabalhando em lugares diferentes e, por vezes, em diferentes fusos horários.
- É praticamente impossível que cada equipe tenha uma visão de todo o sistema.
- Consequentemente, suas prioridades costumam ser voltadas para completar sua parte do sistema, sem levar em conta questões mais amplas do sistema como um todo.

2 - Escalamento de Métodos Ágeis – Grandes Sistemas

- Sistemas de grande porte **incluem e interagem** com **inúmeros sistemas existentes**.
- Muitos dos requisitos do sistema **estão preocupados** com **essa interação**; assim, **realmente não se prestam à flexibilidade** e desenvolvimento incremental.
- Questões políticas também **podem ser importantes** aqui.
- Muitas vezes, **a solução mais fácil** para **um problema** é mudar **um sistema em vigor**.
- No entanto, isso **requer uma negociação** com **os gerentes do sistema** para **convencê-los** de que as **mudanças podem ser implementadas sem risco** para a operação do sistema.

3 - Escalamamento de Métodos Ágeis – Grandes Sistemas

- Sempre que **vários sistemas** estão **integrados** para **criar um único**
- Uma **fração significativa** do desenvolvimento **preocupa-se** com a **configuração** do **sistema** e não **com** o **desenvolvimento** do código original.
- Essa atividade **não é** necessariamente **compatível** com o **desenvolvimento incremental** e com a **integração frequente de sistemas**.

4 - Escalamento de Métodos Ágeis – Grandes Sistemas

- Sistemas de grande porte e seus processos de desenvolvimento são frequentemente restringidos pelas regras externas e regulamentos que limitam o desenvolvimento, que exigem certos tipos de documentação a ser produzida etc.
- Esses sistemas possuem um longo tempo de aquisição e desenvolvimento.
- É difícil manter equipes coerentes que saibam sobre o sistema durante esse período, pois as pessoas, inevitavelmente, deslocam-se para outros trabalhos e projetos

5 - Escalamento de Métodos Ágeis – Grandes Sistemas

- Esses sistemas **possuem diversos stakeholders**.
 - **Por exemplo**, **enfermeiros** e **administradores** podem ser os usuários finais de um sistema médico, mas o **peçoal médico sênior**, **gerentes de hospital** etc. também são stakeholders do sistema.
- É **praticamente impossível envolver**, no processo de desenvolvimento, **todos** esses diferentes stakeholders.

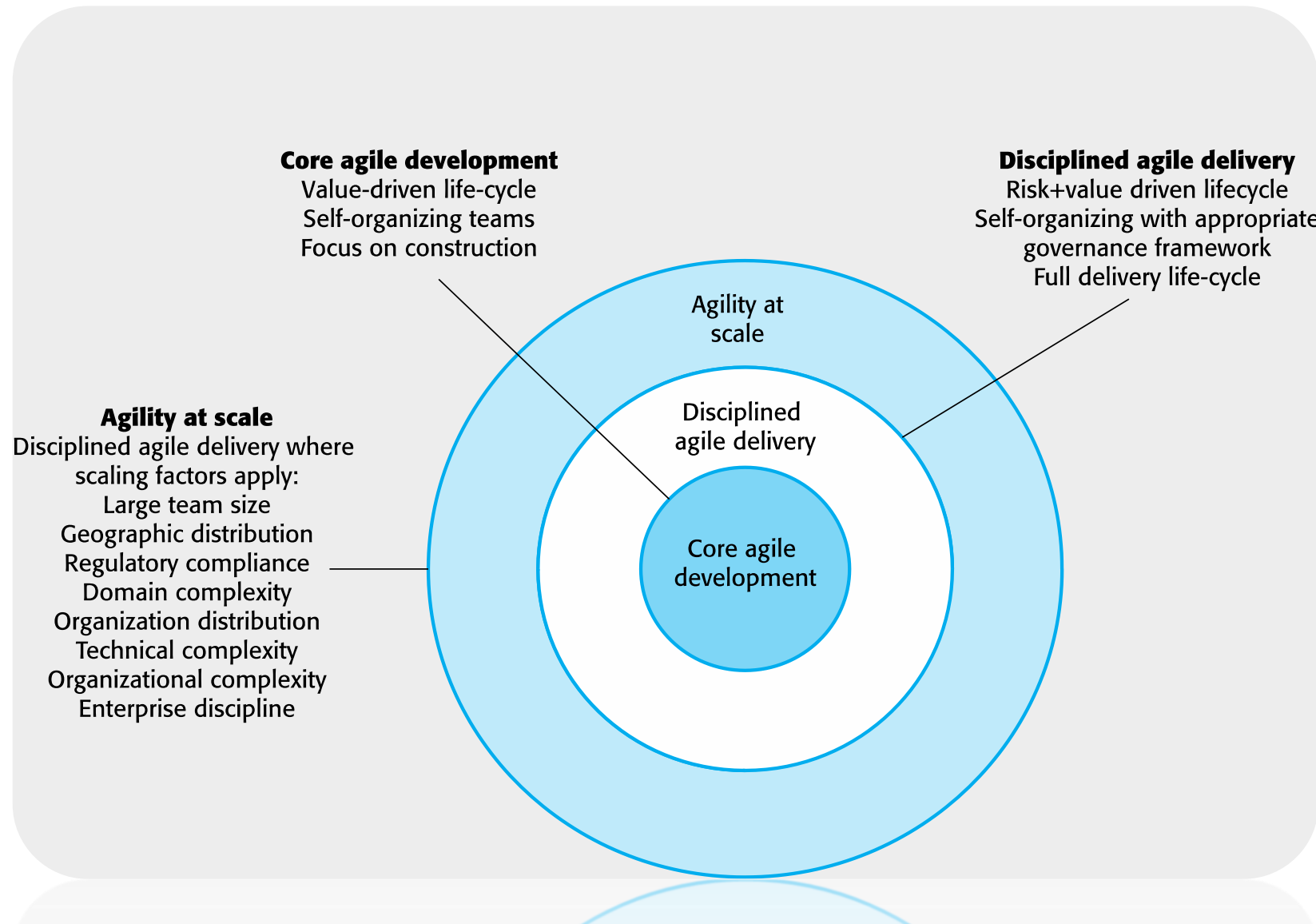
Escalando e Ampliando

- Há duas perspectivas no escalamento de métodos ágeis:
 - **Perspectiva ‘*scaling up*’,**
 - relacionada ao uso desses métodos para **desenvolver sistemas** de software de **grande porte** que **não podem ser desenvolvidos** por uma equipe pequena.
 - **Perspectiva ‘*scaling out*’,**
 - relacionada **com a forma** como **os métodos ágeis** podem **ser introduzidos** em **uma grande organização** com **muitos anos de experiência** em desenvolvimento de software.

Problemas Práticos com Métodos Ágeis

- A informalidade do desenvolvimento ágil é incompatível com a abordagem jurídica para a definição de contratos comumente usada em grandes empresas.
- Os métodos ágeis são mais apropriados para o desenvolvimento de novos softwares do que para a manutenção de softwares.
- No entanto, a maioria dos custos de software em grandes empresas vem da manutenção de seus sistemas de software existentes.
- Os métodos ágeis são projetados para pequenas equipes juntas no mesmo local, embora muito do desenvolvimento de software agora envolva equipes distribuídas em todo o mundo.

Agilidade da IBM um modelo em escala

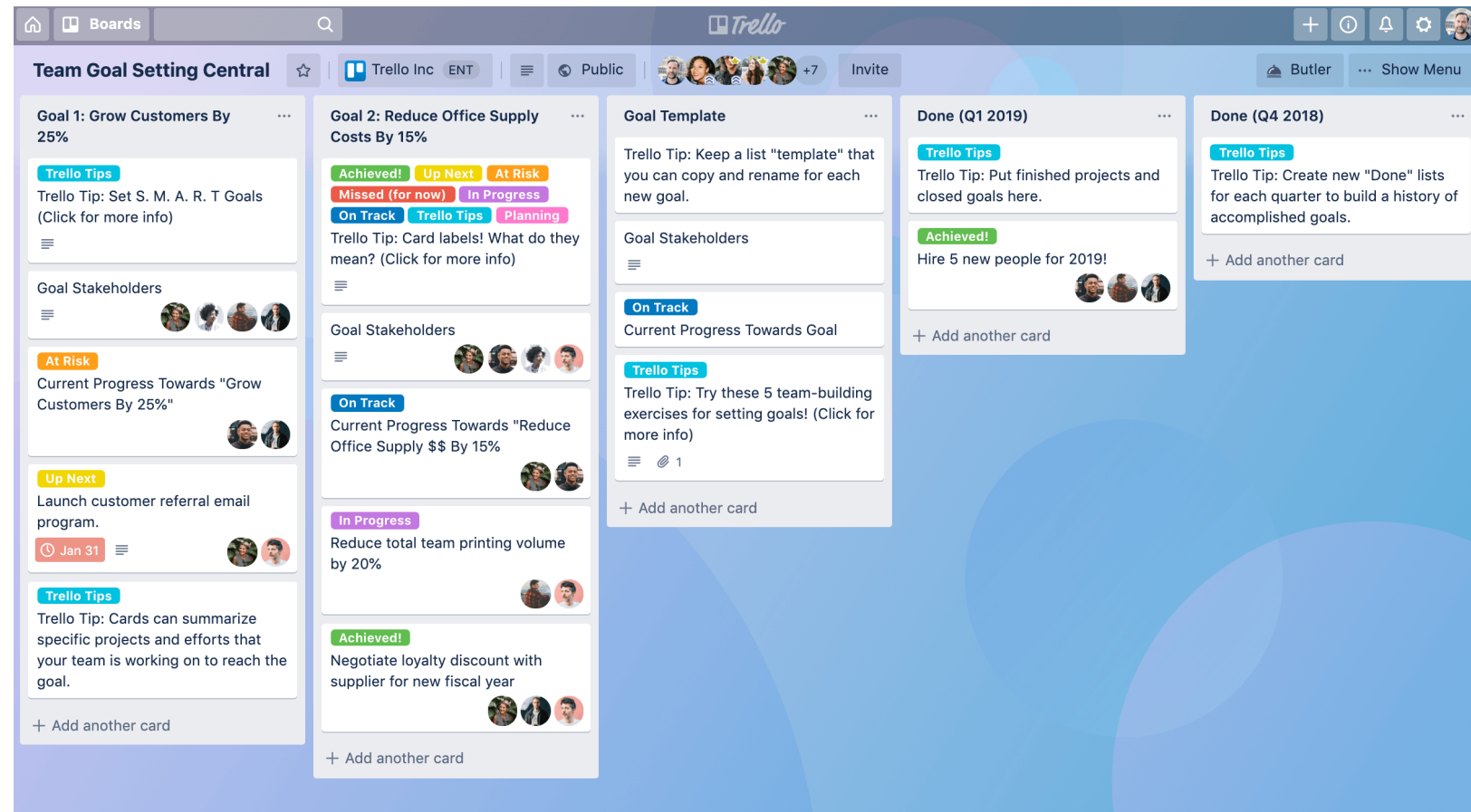


Questões Contratuais

- A maioria dos contratos de software para sistemas customizados é baseada em uma especificação, que define o que deve ser implementado pelo desenvolvedor do sistema para o cliente do sistema.
- No entanto, isso impede a especificação e o desenvolvimento de intercalação, como é feito em desenvolvimento ágil.
- É necessário um contrato que pague pelo tempo do desenvolvedor em vez de pagar por funcionalidade.
- Essa forma de pagamento é visto como um alto risco para muitos departamentos jurídicos, porque o que tem que ser entregue não pode ser garantido por contrato.

Sugestões de Ferramentas

Sugestões de Ferramentas



Sugestões de Ferramentas



A screenshot of the Bitrix24 Planner interface. The top navigation bar includes the Bitrix24 logo, a search bar, the time 5:29 PM, and the user name Heidi Ling. Below the navigation bar, there are tabs for "All", "Ongoing", "Assisting", "Efficiency", "Kanban", and "Projects". The main area is titled "Planner" and shows a grid of tasks categorized by status: "Not planned", "To be done this week", "High Priority", and "Completed". Each task card includes a title, date, and progress indicator. The left sidebar lists various application modules like Activity Stream, Chat and Calls, Workgroups, Calendar, CRM, Open Channels, Sites, CRM Marketing, Bitrix24 Drive, Company, Time and Reports, Applications, Bitrix24 Website, Settings, and More... The right sidebar shows a list of users and a chat icon.

Sugestões de Ferramentas



Projects / Beyond Gravity
Board

Search: [] | [User Avatars] +3 | Epic ▾ | GROUP BY: Choices ▾

TO DO 12	IN PROGRESS 4	IN QA 4	DONE 4
Implement feedback collector NUC-205 9 [User]	Update T&C copy with v1.9 from the writers guild in all products that have cross country compliance NUC-213 1 [User]	Adapt web app no new payments provider NUC-346 5 [User]	Quick booking for accomodations - web NUC-336 4 [User]
Bump version for new API for billing NUC-206 3 [User]	Bump feedback icon version NUC-214 3 [User]	Purchasing error - edit fields NUC-354 3 [User]	Fluid booking on tablets NUC-343 5 [User]
Add NPS feedback to wallboard NUC-208 1 [User]	Tech spike on new stripe integration with paypal NUC-215 3 [User]	Multi-dest search UI web NUC-338 5 [User]	Shoping cart purchasing error - quick fix required. NUC-354 1 [User]
Add analytics events to pricing page NUC-209 3 [User]	Change phone number field type to 'phone' NUC-217 1 [User]		
Resize the images for the upcoming campaign NUC-210 1 [User]			

Referências

1. Sommerville, I.; Engenharia de Software. 10ed, Ed. Pearson, 2019. ISBN: 9788543024974.
2. Wazlawick, R.S., Análise e Projeto de Sistemas de Informação Orientados a Objetos, GEN LTC, 3ª edição, 2014.
3. Guedes, G. T. A. UML 2 - Uma Abordagem Prática – 3ed. Novatec, 2018. ISBN: 9788575226469
4. Cockburn, A. Escrevendo Casos de Uso Eficazes: Um Guia Prático para Desenvolvedores de Software, Bookman, 2005.